🐳 Go Docker 沙箱环境开发学习指南

从零开始掌握使用 Go 语言操作 Docker SDK 构建容器化沙箱环境

版本：1.0.0 | 更新日期：2026-08-17

📑 目录

1. 学习概述与前置准备
2. 阶段一：Docker SDK 基础（第 1-2 天）
3. 阶段二：容器生命周期管理（第 3-4 天）
4. 阶段三：网络与端口管理（第 5-6 天）
5. 阶段四：存储与卷管理（第 7-8 天）
6. 阶段五：镜像构建与管理（第 9-10 天）
7. 阶段六：高级特性（第 11-14 天）
8. 阶段七：综合实战项目（第 15-21 天）
9. 参考资源

📋 学习概述与前置准备

学习目标

通过 21 天的系统学习，掌握使用 Go 语言开发 Docker 沙箱环境的核心技能：

- 理解 Docker 核心概念（容器、镜像、网络、卷）
- 熟练使用 Go Docker SDK 进行容器生命周期管理

- 实现容器网络隔离与端口映射
- 掌握镜像构建与管理
- 构建完整的沙箱编排系统

前置知识

技能	要求程度	学习资源
Go 语言基础	熟练	掌握 goroutine、channel、interface、error handling
Docker 基础	了解	会使用 docker run/build/ps 等基本命令
HTTP/REST API	了解	理解请求/响应、状态码、JSON
Linux 基础	了解	文件系统、进程、网络基础概念

环境准备

```
# 1. 安装 Go（1.21+）
go version

# 2. 安装 Docker Desktop / Docker Engine
docker version

# 3. 创建学习项目
mkdir go-docker-sandbox-lab && cd go-docker-sandbox-lab
go mod init go-docker-sandbox-lab

# 4. 安装 Docker SDK
go get github.com/docker/docker/client
go get github.com/docker/docker/api/types
go get github.com/docker/go-connections/nat
```

项目结构参考

```
go-docker-sandbox-lab/
├── cmd/
│   └── lab/
│       └── main.go           # 实验入口
├── internal/
│   ├── docker/
│   │   ├── client.go         # Docker 客户端封装
│   │   ├── container.go      # 容器操作
│   │   ├── network.go        # 网络管理
│   │   ├── volume.go         # 卷管理
│   │   └── image.go          # 镜像管理
│   ├── sandbox/
│   │   ├── orchestrator.go   # 沙箱编排器
│   │   ├── registry.go       # 实例注册表
│   │   └── health.go         # 健康检查
│   └── config/
│       └── config.go         # 配置管理
├── config/
│   └── config.yaml           # 配置文件
├── go.mod
└── go.sum
```

🚀 阶段一：Docker SDK 基础（第 1-2 天）

Day 1-2

1.1 学习 Docker SDK 客户端初始化

📖 知识点：Docker Client 创建

Docker Go SDK 的核心是 `client.Client` ，它封装了与 Docker daemon 的通信。

```
package main

import (
    "context"
    "fmt"
    "log"

    "github.com/docker/docker/client"
)

func main() {
    // 方式1：使用默认配置（读取 DOCKER_HOST 环境变量）
    cli, err := client.NewClientWithOpts(client.FromEnv)
    if err != nil {
        log.Fatal(err)
    }
    defer cli.Close()

    // 方式2：显式指定 Docker host
    cli2, err := client.NewClientWithOpts(
        client.WithHost("tcp://localhost:2375"),
    )
    if err != nil {
        log.Fatal(err)
    }
    defer cli2.Close()

    // 方式3：禁用代理（内网/本机 Docker 推荐）
    transport := http.DefaultTransport.(*http.Transport).Clone()
```

```
    transport.Proxy = nil
    transport.TLSClientConfig = nil // 强制 HTTP
    cli3, err := client.NewClientWithOpts(
        client.WithHTTPClient(&http.Client{Transport: transport}),
    )
    if err != nil {
        log.Fatal(err)
    }
    defer cli3.Close()

    // 验证连接
    info, err := cli.Ping(context.Background())
    if err != nil {
        log.Fatal("无法连接 Docker:", err)
    }
    fmt.Println("Docker API 版本:", info.APIVersion)
}
```

> 💡 **提示：**生产环境建议禁用 HTTP 代理（Docker daemon 通常是内网/本机地址），并根据实际情况选择连接方式。

1.2 实践任务

任务 1：创建 Docker 客户端封装　入门　⏱ 2小时

> 🎯 **目标：**创建一个可复用的 Docker 客户端封装，支持配置化初始化。

要求：

1. 创建 `internal/docker/client.go`
2. 实现 `DockerConfig` 结构体，支持配置 Host、代理、超时
3. 实现 `NewDockerClient(cfg *DockerConfig) (*client.Client, error)`
4. 处理常见错误：Docker 未启动、权限不足、连接超时

```
// 参考实现框架
type DockerConfig struct {
    Host        string        // Docker host 地址
    DisableProxy bool         // 是否禁用 HTTP 代理
    Timeout     time.Duration // 连接超时
}

func NewDockerClient(cfg DockerConfig) (*client.Client, error) {
    var opts []client.Opt

    if cfg.Host != "" {
        opts = append(opts, client.WithHost(cfg.Host))
    } else {
        opts = append(opts, client.FromEnv)
    }

    // TODO: 配置传输层（禁用代理等）
    // TODO: 创建并返回 client
    // TODO: Ping 验证连接
}
```

> ✅ **检查点：**
>
> - 能成功连接本地 Docker daemon
> - 能正确处理 Docker 未启动的情况
> - 能配置禁用 HTTP 代理

任务 2：实现 Docker 信息查询　入门　⏱ 1小时

> 🎯 **目标：**查询 Docker 系统信息，熟悉 SDK 的查询 API。

要求：

1. 查询 Docker 系统信息： `cli.Info(ctx)`
2. 查询 Docker 版本： `cli.ServerVersion(ctx)`

3. 列出所有容器： `cli.ContainerList(ctx, types.ContainerListOptions{})`

4. 列出所有镜像： `cli.ImageList(ctx, types.ImageListOptions{})`

```
func ShowDockerInfo(ctx context.Context, cli *client.Client) error {
    // 1. 系统信息
    info, err := cli.Info(ctx)
    if err != nil {
        return fmt.Errorf("获取系统信息失败: %w", err)
    }
    fmt.Printf("OS: %s\n", info.OperatingSystem)
    fmt.Printf("Containers: %d\n", info.Containers)
    fmt.Printf("Images: %d\n", info.Images)

    // 2. 版本信息
    ver, err := cli.ServerVersion(ctx)
    if err != nil {
        return fmt.Errorf("获取版本失败: %w", err)
    }
    fmt.Printf("Docker Version: %s\n", ver.Version)

    // 3. 容器列表
    containers, err := cli.ContainerList(ctx, container.ListOptions{
        All: true,
    })
    // TODO: 格式化输出容器信息

    return nil
}
```

🔧 阶段二：容器生命周期管理（第 3-4 天）

Day 3-4

2.1 容器创建与配置

📖 知识点：ContainerCreate 核心参数

Docker 容器创建需要三个核心配置：

配置类型	结构体	作用
容器配置	`container.Config`	镜像、环境变量、暴露端口等
主机配置	`container.HostConfig`	端口映射、卷挂载、资源限制
网络配置	`network.NetworkingConfig`	网络端点配置

```
// 创建容器示例
func CreateSandbox(ctx context.Context, cli *client.Client) (string, error) {
    // 1. 容器配置
    containerCfg := &container.Config{
        Image: "ubuntu:22.04",
        Env: []string{
            "SANDBOX_API_KEY=your-key",
            "TZ=Asia/Shanghai",
        },
        ExposedPorts: nat.PortSet{
            "8080/tcp": {},
        },
        Cmd: []string{"/bin/bash"},
    }

    // 2. 主机配置
    hostCfg := &container.HostConfig{
```

```
            PortBindings: nat.PortMap{
                "8080/tcp": []nat.PortBinding{
                    {
                        HostIP:   "0.0.0.0",
                        HostPort: "18080",
                    },
                },
            },
            // 资源限制
            Resources: container.Resources{
                Memory:   512 * 1024 * 1024, // 512MB
                NanoCPUs: 1e9,               // 1 CPU
            },
            // 安全选项
            SecurityOpt: []string{"seccomp=unconfined"},
        }

        // 3. 创建容器
        resp, err := cli.ContainerCreate(
            ctx,
            containerCfg,
            hostCfg,
            nil, // 网络配置
            nil, // 平台配置
            "my-sandbox-001", // 容器名
        )
        if err != nil {
            return "", fmt.Errorf("创建容器失败: %w", err)
        }

        return resp.ID, nil
    }
```

2.2 容器生命周期操作

任务 3：实现完整的容器生命周期 进阶 ⏱ 4小时

> 🎯 **目标：**实现容器的创建、启动、停止、删除、重启全流程。

核心 API：

```
// 启动容器
err := cli.ContainerStart(ctx, containerID, container.StartOptions{})

// 停止容器
timeout := 10
err := cli.ContainerStop(ctx, containerID, container.StopOptions{
    Timeout: &timeout,
})

// 重启容器
err := cli.ContainerRestart(ctx, containerID, container.StopOptions{})

// 删除容器
err := cli.ContainerRemove(ctx, containerID, container.RemoveOptions{
    Force:         true,  // 强制删除运行中的容器
    RemoveVolumes: false, // 是否删除关联卷
})

// 获取容器状态
inspect, err := cli.ContainerInspect(ctx, containerID)
fmt.Println("Status:", inspect.State.Status)
fmt.Println("Running:", inspect.State.Running)
```

实现要求：

1. 创建 `internal/docker/container.go`
2. 实现 `ContainerManager` 结构体
3. 实现以下方法：
   - `Create(opts CreateOptions) (containerID, error)`
   - `Start(containerID) error`
   - `Stop(containerID) error`
   - `Remove(containerID, force) error`
   - `Inspect(containerID) (*InspectResponse, error)`
   - `List(all bool) ([]ContainerSummary, error)`
4. 每个方法都要处理错误并返回中文错误信息

> 💡 **提示：**实现时注意回滚机制——任何步骤失败都要清理已创建的资源（容器、卷、端口等），避免资源泄漏。

2.3 错误处理与幂等性

📖 知识点：Docker SDK 错误处理

```
import (
    "github.com/docker/docker/client"
    "github.com/containerd/errdefs"
)

func SafeRemove(ctx context.Context, cli *client.Client, containerID string) error {
    err := cli.ContainerRemove(ctx, containerID, container.RemoveOptions{
        Force: true,
    })
    if err != nil {
        // 检查是否是"未找到"错误（幂等处理）
        if client.IsErrNotFound(err) {
            return nil // 容器已不存在，视为成功
        }
        // 检查是否是冲突错误（容器正在运行）
        if errdefs.IsConflict(err) {
            // 先停止再删除
            timeout := 10
            _ = cli.ContainerStop(ctx, containerID, container.StopOptions{
                Timeout: &timeout,
            })
            return cli.ContainerRemove(ctx, containerID, container.RemoveOptions{
                Force: true,
            })
        }
        return fmt.Errorf("删除容器失败: %w", err)
    }
    return nil
}
```

> ✅ **检查点：**

> - 删除不存在的容器不会报错（幂等）
> - 能正确处理容器正在运行的情况
> - 错误信息使用中文且包含上下文

🌐 阶段三：网络与端口管理（第 5-6 天）

Day 5-6

3.1 Docker 网络管理

📖 知识点：网络隔离策略

沙箱环境需要网络隔离，常见策略：

策略	说明	适用场景
per_user	每个用户一个独立网络	用户间完全隔离
per_scene	每个场景一个共享网络	同场景用户可互访
bridge	默认桥接网络	开发测试

```
// 创建网络
func CreateNetwork(ctx context.Context, cli *client.Client, name string) (string, erro
    resp, err := cli.NetworkCreate(ctx, name, network.CreateOptions{
        Driver: "bridge",
        Labels: map[string]string{
            "managed-by": "sandbox-orchestrator",
        },
    })
    if err != nil {
        return "", fmt.Errorf("创建网络失败: %w", err)
    }
    return resp.ID, nil
}

// 连接容器到网络
func ConnectToNetwork(ctx context.Context, cli *client.Client, networkID, containerID s
    return cli.NetworkConnect(ctx, networkID, containerID, &network.EndpointSettings{
```

```
        // 可选：指定容器在网络中的 IP
        // IPAddress: "172.20.0.10",
    })
}

// 断开网络
func DisconnectFromNetwork(ctx context.Context, cli *client.Client, networkID, containe
    return cli.NetworkDisconnect(ctx, networkID, containerID, true)
}

// 删除网络
func RemoveNetwork(ctx context.Context, cli *client.Client, networkID string) error {
    return cli.NetworkRemove(ctx, networkID)
}

// 列出网络
func ListNetworks(ctx context.Context, cli *client.Client) ([]network.Inspect, error) 
    return cli.NetworkList(ctx, network.ListOptions{})
}
```

3.2 端口管理

任务 4：实现端口池管理 进阶 ⏱ 3小时

> 🎯 **目标：**实现一个端口池，支持动态分配和回收宿主机端口。

参考实现：

```
// PortPool 端口池：管理宿主机可用端口范围
type PortPool struct {
    mu      sync.Mutex
    start   int
    end     int
    used    map[int]bool
}

func NewPortPool(start, end int) *PortPool {
    return &PortPool{
        start: start,
        end:   end,
        used:  make(map[int]bool),
```

```
    }
}

// Allocate 分配一个可用端口
func (p *PortPool) Allocate() (int, error) {
    p.mu.Lock()
    defer p.mu.Unlock()

    for port := p.start; port <= p.end; port++ {
        if !p.used[port] && isPortAvailable(port) {
            p.used[port] = true
            return port, nil
        }
    }
    return 0, fmt.Errorf("端口池已耗尽（范围 %d-%d）", p.start, p.end)
}

// Release 回收端口
func (p *PortPool) Release(port int) {
    p.mu.Lock()
    defer p.mu.Unlock()
    delete(p.used, port)
}

// isPortAvailable 检查端口是否被占用
func isPortAvailable(port int) bool {
    ln, err := net.Listen("tcp", fmt.Sprintf(":%d", port))
    if err != nil {
        return false
    }
    ln.Close()
    return true
}

// UsedCount 当前已分配端口数
func (p *PortPool) UsedCount() int {
    p.mu.Lock()
    defer p.mu.Unlock()
    return len(p.used)
}
```

> ✅ **检查点：**
>
> - 能分配和回收端口
> - 并发安全（使用 mutex）

- 能检测端口是否被占用
- 端口池耗尽时返回明确错误

3.3 网络隔离实战

任务 5：实现用户网络隔离 挑战 ⏱ 4小时

> 🎯 **目标：**实现 per_user 网络隔离，每个用户的容器在独立网络中。

实现要求：

1. 为每个用户创建独立的 Docker 网络
2. 网络命名规范： `{prefix}-net-{userId}`
3. 用户所有容器加入同一网络（可互访）
4. 不同用户网络完全隔离
5. 用户无活跃容器时自动清理网络

```
// 网络命名
func userNetworkName(prefix, userID string) string {
    return fmt.Sprintf("%s-net-%s", prefix, sanitize(userID))
}

// 确保用户网络存在（幂等）
func (o *Orchestrator) ensureUserNetwork(ctx context.Context, userID string) error {
    name := userNetworkName(o.cfg.Prefix, userID)

    // 检查是否已存在
    if _, err := o.cli.NetworkInspect(ctx, name, network.InspectOptions{}); err == nil
        return nil
    }

    // 创建网络
    _, err := o.cli.NetworkCreate(ctx, name, network.CreateOptions{
        Driver: "bridge",
        Labels: map[string]string{
```

```
            "sandbox.user_id": userID,
            "sandbox.managed": "true",
        },
    })
    if err != nil {
        // 并发创建竞态：检查是否已被其他 goroutine 创建
        if _, ierr := o.cli.NetworkInspect(ctx, name, network.InspectOptions{}); ierr =
            return nil
        }
        return fmt.Errorf("创建用户网络失败: %w", err)
    }
    return nil
}

// 释放空闲用户网络
func (o *Orchestrator) releaseUserNetworkIfIdle(ctx context.Context, userID string) {
    if o.hasActiveContainers(userID) {
        return // 用户还有活跃容器
    }
    name := userNetworkName(o.cfg.Prefix, userID)
    _ = o.cli.NetworkRemove(ctx, name)
}
```

💾 阶段四：存储与卷管理（第 7-8 天）

Day 7-8

4.1 Docker 卷管理

📖 知识点：卷 vs 绑定挂载

类型	特点	适用场景
Volume（卷）	Docker 管理，独立于容器生命周期	持久化数据、跨容器共享
Bind Mount	直接挂载宿主机目录	开发环境、配置文件
tmpfs	内存临时文件系统	敏感数据、临时缓存

```
// 创建卷
func CreateVolume(ctx context.Context, cli *client.Client, name string) error {
    _, err := cli.VolumeCreate(ctx, volume.CreateOptions{
        Name: name,
        Labels: map[string]string{
            "sandbox.workspace": "true",
        },
    })
    return err
}

// 删除卷
func RemoveVolume(ctx context.Context, cli *client.Client, name string) error {
    return cli.VolumeRemove(ctx, name, true) // true = force
}

// 列出卷
func ListVolumes(ctx context.Context, cli *client.Client) ([]*volume.Volume, error) {
    resp, err := cli.VolumeList(ctx, volume.ListOptions{})
```

```
    if err != nil {
        return nil, err
    }
    return resp.Volumes, nil
}

// 挂载卷到容器
func WithVolumeMount(volumeName, containerPath string) mount.Mount {
    return mount.Mount{
        Type:   mount.TypeVolume,
        Source: volumeName,
        Target: containerPath,
    }
}

// 绑定挂载
func WithBindMount(hostPath, containerPath string) mount.Mount {
    return mount.Mount{
        Type:     mount.TypeBind,
        Source:   hostPath,
        Target:   containerPath,
        ReadOnly: false,
    }
}
```

4.2 Workspace 持久化

任务 6：实现 Workspace 卷管理 进阶 ⏱ 3小时

> 🎯 **目标：**实现沙箱 workspace 的持久化存储，支持创建、复用、清理。

实现要求：

1. 创建卷时命名规范： `{prefix}-ws-{instanceId}`
2. 支持卷的复用（Restore 时保留数据）
3. 实现卷的清理策略：
   - Destroy 时可选清理卷
   - Restore 时可选保留卷

4. 错误处理：卷不存在、卷被占用等

```
// WorkspaceManager 管理沙箱工作空间卷
type WorkspaceManager struct {
    cli    *client.Client
    prefix string
}

// CreateWorkspace 创建工作空间卷
func (m *WorkspaceManager) CreateWorkspace(ctx context.Context, instanceID string) (str
    volumeName := fmt.Sprintf("%s-ws-%s", m.prefix, instanceID)

    _, err := m.cli.VolumeCreate(ctx, volume.CreateOptions{
        Name: volumeName,
        Labels: map[string]string{
            "sandbox.instance_id": instanceID,
            "sandbox.type":        "workspace",
        },
    })
    if err != nil {
        return "", fmt.Errorf("创建 workspace 卷失败: %w", err)
    }

    return volumeName, nil
}

// RemoveWorkspace 删除工作空间卷
func (m *WorkspaceManager) RemoveWorkspace(ctx context.Context, volumeName string) err
    if volumeName == "" {
        return nil
    }
    err := m.cli.VolumeRemove(ctx, volumeName, true)
    if err != nil && !client.IsErrNotFound(err) {
        return fmt.Errorf("删除 workspace 卷失败: %w", err)
    }
    return nil
}

// MountWorkspace 挂载工作空间到容器配置
func MountWorkspace(volumeName, containerPath string) mount.Mount {
    return mount.Mount{
        Type:   mount.TypeVolume,
        Source: volumeName,
        Target: containerPath,
```

```
        }
    }
```

🏗️ 阶段五：镜像构建与管理（第 9-10 天）

Day 9-10

5.1 镜像拉取与检查

📖 知识点：镜像基础操作

```
// 拉取镜像
func PullImage(ctx context.Context, cli *client.Client, imageRef string) error {
    reader, err := cli.ImagePull(ctx, imageRef, image.PullOptions{})
    if err != nil {
        return fmt.Errorf("拉取镜像失败: %w", err)
    }
    defer reader.Close()

    // 必须读取完所有数据，否则拉取会中断
    _, err = io.Copy(io.Discard, reader)
    return err
}

// 检查镜像是否存在
func ImageExists(ctx context.Context, cli *client.Client, imageRef string) bool {
    _, err := cli.ImageInspect(ctx, imageRef)
    return err == nil
}

// 确保镜像存在（不存在则拉取）
func EnsureImage(ctx context.Context, cli *client.Client, imageRef string) error {
    if ImageExists(ctx, cli, imageRef) {
        return nil
    }
    return PullImage(ctx, cli, imageRef)
}

// 列出镜像
func ListImages(ctx context.Context, cli *client.Client) ([]image.Summary, error) {
    return cli.ImageList(ctx, image.ListOptions{})
}
```

```
// 删除镜像
func RemoveImage(ctx context.Context, cli *client.Client, imageRef string) error {
    _, err := cli.ImageRemove(ctx, imageRef, image.RemoveOptions{
        Force: true,
    })
    return err
}
```

5.2 动态构建镜像

任务 7：实现镜像构建器 挑战 ⏱️ 5小时

> 🎯 **目标：**实现动态构建 Docker 镜像，支持自定义 Dockerfile 和构建步骤。

核心实现：

```
// ImageBuilder 镜像构建器
type ImageBuilder struct {
    cli        *client.Client
    namePrefix string
}

// BuildOptions 构建选项
type BuildOptions struct {
    ImageName   string            // 目标镜像名
    BaseImage   string            // 基础镜像
    Dockerfile  string            // 自定义 Dockerfile 内容
    BuildSteps  []string          // 构建步骤（生成 RUN 指令）
    Files       map[string][]byte // 预置文件
    EnvVars     map[string]string // 环境变量
    Labels      map[string]string // 镜像标签
}

// Build 构建镜像
func (b *ImageBuilder) Build(ctx context.Context, opts BuildOptions) (string, error) {
    // 1. 生成 Dockerfile
    dockerfile := opts.Dockerfile
    if dockerfile == "" {
        dockerfile = b.generateDockerfile(opts)
    }
```

```
    // 2. 创建构建上下文（tar 包）
    buildCtx, err := b.createBuildContext(dockerfile, opts.Files)
    if err != nil {
        return "", fmt.Errorf("创建构建上下文失败: %w", err)
    }
    defer buildCtx.Close()

    // 3. 执行构建
    imageName := b.namePrefix + opts.ImageName
    resp, err := b.cli.ImageBuild(ctx, buildCtx, types.ImageBuildOptions{
        Tags:        []string{imageName},
        Labels:      opts.Labels,
        Remove:      true,  // 构建后删除中间容器
        ForceRemove: true,
    })
    if err != nil {
        return "", fmt.Errorf("构建镜像失败: %w", err)
    }
    defer resp.Body.Close()

    // 4. 读取构建日志（必须读完，否则构建会中断）
    // 可以解析 JSON 格式的日志获取构建进度
    scanner := bufio.NewScanner(resp.Body)
    for scanner.Scan() {
        var logEntry struct {
            Stream string `json:"stream"`
            Error  string `json:"error"`
        }
        if err := json.Unmarshal(scanner.Bytes(), &logEntry); err == nil {
            if logEntry.Error != "" {
                return "", fmt.Errorf("构建失败: %s", logEntry.Error)
            }
            // 可以输出日志: fmt.Print(logEntry.Stream)
        }
    }

    return imageName, nil
}

// generateDockerfile 根据选项生成 Dockerfile
func (b *ImageBuilder) generateDockerfile(opts BuildOptions) string {
    var buf bytes.Buffer

    // FROM
    base := opts.BaseImage
    if base == "" {
        base = "ubuntu:22.04"
    }
```

```
    fmt.Fprintf(&buf, "FROM %s\n\n", base)

    // ENV
    for k, v := range opts.EnvVars {
        fmt.Fprintf(&buf, "ENV %s=%s\n", k, v)
    }
    if len(opts.EnvVars) > 0 {
        buf.WriteString("\n")
    }

    // COPY files
    for path := range opts.Files {
        fmt.Fprintf(&buf, "COPY %s %s\n", filepath.Base(path), path)
    }

    // RUN steps
    for _, step := range opts.BuildSteps {
        fmt.Fprintf(&buf, "RUN %s\n", step)
    }

    return buf.String()
}

// createBuildContext 创建 tar 格式的构建上下文
func (b *ImageBuilder) createBuildContext(dockerfile string, files map[string][]byte) (
    buf := new(bytes.Buffer)
    tw := tar.NewWriter(buf)
    defer tw.Close()

    // 添加 Dockerfile
    if err := addFileToTar(tw, "Dockerfile", []byte(dockerfile)); err != nil {
        return nil, err
    }

    // 添加额外文件
    for path, content := range files {
        if err := addFileToTar(tw, filepath.Base(path), content); err != nil {
            return nil, err
        }
    }

    return io.NopCloser(buf), nil
}
```

> ✅ **检查点：**
>
> - 能从基础镜像构建新镜像

- 能注入自定义文件
- 能设置环境变量和标签
- 构建失败能返回详细错误

⚡ 阶段六：高级特性（第 11-14 天）

Day 11-14

6.1 资源限制

📖 知识点：容器资源控制

```
// 资源限制配置
func WithResourceLimits(memoryMB int64, cpuCount float64) container.Resources {
    resources := container.Resources{}

    // 内存限制（字节）
    if memoryMB > 0 {
        resources.Memory = memoryMB * 1024 * 1024
        // 可选：设置内存 swap 限制
        // resources.MemorySwap = memoryMB * 2 * 1024 * 1024
    }

    // CPU 限制（纳核，支持小数）
    if cpuCount > 0 {
        resources.NanoCPUs = int64(cpuCount * 1e9)
    }

    return resources
}

// 使用示例
hostCfg := &container.HostConfig{
    Resources: WithResourceLimits(512, 1.5), // 512MB 内存，1.5 CPU
}
```

6.2 健康检查

任务 8：实现容器健康检查 进阶 ⏱ 3小时

> 🎯 **目标：**实现等待容器服务就绪的健康检查机制。

```
// HealthChecker 健康检查器
type HealthChecker struct {
    client  *http.Client
    timeout time.Duration
}

// WaitUntilReady 等待服务就绪
func (h *HealthChecker) WaitUntilReady(ctx context.Context, baseURL, apiKey string) er
    healthURL := baseURL + "/health"

    deadline := time.After(h.timeout)
    ticker := time.NewTicker(500 * time.Millisecond)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-deadline:
            return fmt.Errorf("健康检查超时（%v）", h.timeout)
        case <-ticker.C:
            req, _ := http.NewRequestWithContext(ctx, "GET", healthURL, nil)
            if apiKey != "" {
                req.Header.Set("Authorization", "Bearer "+apiKey)
            }

            resp, err := h.client.Do(req)
            if err != nil {
                continue // 连接失败，继续重试
            }
            resp.Body.Close()

            if resp.StatusCode == http.StatusOK {
                return nil // 服务就绪
            }
        }
    }
}

// 使用示例
checker := &HealthChecker{
```

```
    client:  &http.Client{Timeout: 2 * time.Second},
    timeout: 60 * time.Second,
}
err := checker.WaitUntilReady(ctx, "http://localhost:18080", "your-api-key")
if err != nil {
    log.Fatal("服务未就绪:", err)
}
```

6.3 容器日志与监控

任务 9：实现容器日志收集 进阶 ⏱ 2小时

> 🎯 **目标：**实现容器日志的实时读取和历史查询。

```
// 获取容器日志
func GetContainerLogs(ctx context.Context, cli *client.Client, containerID string, foll
    return cli.ContainerLogs(ctx, containerID, container.LogsOptions{
        ShowStdout: true,
        ShowStderr: true,
        Follow:     follow,    // 实时跟踪
        Tail:       "100",     // 最近 100 行
        Timestamps: true,      // 显示时间戳
    })
}

// 使用示例
logs, err := GetContainerLogs(ctx, cli, containerID, true)
if err != nil {
    log.Fatal(err)
}
defer logs.Close()

// 日志是 multiplexed 格式，需要处理 header
// 或者使用 stdcopy 分离 stdout/stderr
stdcopy.StdCopy(os.Stdout, os.Stderr, logs)
```

6.4 并发控制

📖 知识点：创建并发限流

```
// Orchestrator 使用信号量限制并发创建
type Orchestrator struct {
    createSem chan struct{} // 并发信号量
}

func NewOrchestrator(maxConcurrent int) *Orchestrator {
    return &Orchestrator{
        createSem: make(chan struct{}, maxConcurrent),
    }
}

// Create 创建沙箱（带并发限制）
func (o *Orchestrator) Create(ctx context.Context, opts CreateOptions) error {
    // 尝试获取信号量
    select {
    case o.createSem <- struct{}{}:
        defer func() { <-o.createSem }() // 释放
    default:
        return fmt.Errorf("创建并发已达上限，请稍后重试")
    }

    // 执行创建逻辑...
    return nil
}
```

🎯 阶段七：综合实战项目（第 15-21 天）

Day 15-21

7.1 项目目标

构建一个完整的沙箱编排系统，具备以下能力：

1. **生命周期管理**：创建、启动、停止、销毁、还原
2. **网络隔离**：per_user 网络隔离
3. **存储管理**：workspace 持久化
4. **资源控制**：内存、CPU 限制
5. **健康检查**：等待服务就绪
6. **实例注册**：管理所有沙箱实例

7.2 架构设计

```
// SandboxOrchestrator 沙箱编排器
type SandboxOrchestrator struct {
    docker       *client.Client
    config       *Config
    portPool     *PortPool
    registry     *InstanceRegistry
    healthCheck  *HealthChecker
    workspace    *WorkspaceManager
    createSem    chan struct{}
}

// InstanceEntry 实例注册条目
type InstanceEntry struct {
    InstanceID  string
    ContainerID string
    BaseURL     string
    HostPort    int
    VolumeName  string
```

```
    UserID      string
    SceneID     string
    CreateTime  time.Time
    Status      string
}

// InstanceRegistry 实例注册表
type InstanceRegistry struct {
    mu       sync.RWMutex
    entries  map[string]*InstanceEntry
}
```

7.3 实战任务清单

任务 10：实现完整沙箱编排器 挑战 ⏱ 2天

> 🎯 **目标：**实现一个完整的沙箱编排器。

实现清单：

功能	方法	优先级
创建沙箱	`Create(ctx, opts) (*Result, error)`	⭐⭐⭐
销毁沙箱	`Destroy(ctx, instanceID) error`	⭐⭐⭐
启动沙箱	`Start(ctx, instanceID) error`	⭐⭐
停止沙箱	`Stop(ctx, instanceID) error`	⭐⭐

还原沙箱	`Restore(ctx, instanceID, keepWS) (*Result, error)`	⭐⭐
查询状态	`Inspect(ctx, instanceID) (*Info, error)`	⭐⭐
实例列表	`List() []*InstanceEntry`	⭐

> 💡 **关键实现要点：**
>
> - 实现回滚机制：任何步骤失败都要清理已创建的资源
> - 使用信号量限制并发创建
> - 网络模式校验：防止逃逸（禁止 host/none/bridge）
> - 容器命名规范：`{prefix}-{sceneId}-{userId}`
> - 实例注册表：记录所有活跃沙箱实例的状态

任务 11：实现 Restore 还原功能　挑战　⏱ 1天

> 🎯 **目标：**实现沙箱还原功能，销毁后用相同参数重新创建。

核心逻辑：

1. 获取原实例信息
2. 移除旧容器（幂等）
3. 释放端口、摘除注册条目
4. 用相同 instance_id 重新创建
    - keepWorkspace=true → 复用旧卷

- keepWorkspace=false → 新建空卷

5. 失败回滚：占回原端口、恢复注册条目

> ⚠️ **注意：**还原后 instance_id 保持不变，平台侧引用不会断开。

📚 参考资源

官方文档

- **Docker SDK for Go**：pkg.go.dev/github.com/docker/docker/client
- **Docker Engine API**：docs.docker.com/engine/api
- **Docker 官方教程**：docs.docker.com/develop/sdk

常用 API 速查

操作	API
创建容器	`cli.ContainerCreate(ctx, config, hostConfig, networkConfig, platform, name)`
启动容器	`cli.ContainerStart(ctx, containerID, options)`
停止容器	`cli.ContainerStop(ctx, containerID, options)`
删除容器	`cli.ContainerRemove(ctx, containerID, options)`
检查容器	`cli.ContainerInspect(ctx, containerID)`
容器列表	`cli.ContainerList(ctx, options)`

创建网络	`cli.NetworkCreate(ctx, name, options)`
创建卷	`cli.VolumeCreate(ctx, options)`
拉取镜像	`cli.ImagePull(ctx, ref, options)`
构建镜像	`cli.ImageBuild(ctx, context, options)`

学习路径总结

```
Week 1: 基础
├── Day 1-2: Docker SDK 客户端、基础查询
├── Day 3-4: 容器生命周期管理
└── Day 5-6: 网络与端口管理

Week 2: 进阶
├── Day 7-8: 存储与卷管理
├── Day 9-10: 镜像构建与管理
└── Day 11-14: 高级特性（资源限制、健康检查、并发控制）

Week 3: 实战
├── Day 15-17: 完整编排器实现
├── Day 18-19: Restore/回滚机制
└── Day 20-21: 测试与优化
```

🎉 学习完成！

完成所有任务后，你将具备构建生产级 Docker 沙箱编排系统的能力。

—— Go Docker 沙箱环境开发学习指南 v1.0.0 ——

Generated: 2026-08-17